

Python - Chapitre 2

Fonctions, collections et programmes fiables

Quiz de révision | 10 questions | Une seule bonne réponse par question

Question 1

Qu'affiche la dernière ligne de ce programme ?

```
def double(n):  
    print(2 * n)  
  
r = double(3)  
print(r)
```

- A. 6
- B. None
- C. 9
- D. Le programme provoque une erreur

Bonne réponse : B. None

L'appel affiche d'abord 6. Sans return, la fonction renvoie None : c'est la valeur de r, affichée par la dernière ligne.

Question 2

Quelle valeur est affichée ?

```
def bornes(valeurs):  
    return min(valeurs), max(valeurs)  
  
bas, haut = bornes([7, 2, 9])  
print(haut - bas)
```

- A. (2, 9)
- B. 9
- C. 11
- D. 7

Bonne réponse : D. 7

La fonction renvoie le tuple (2, 9). Le déballage donne bas = 2 et haut = 9 ; la différence vaut 7.

Python - Chapitre 2

Fonctions, collections et programmes fiables

Quiz de révision | 10 questions | Une seule bonne réponse par question

Question 3

Quelle est la longueur de a après cet ajout ?

```
a = [1, 2]
a.append([3, 4])
print(len(a))
```

- A. 3
- B. 4
- C. 2
- D. None

Bonne réponse : A. 3

append ajoute un seul objet. La liste devient [1, 2, [3, 4]] : elle contient trois éléments. Avec extend, elle en contiendrait quatre.

Question 4

Que contient a à la fin ?

```
a = [10, 20]
b = a
c = a.copy()
b.append(30)
c[0] = 99
print(a)
```

- A. [10, 20]
- B. [99, 20, 30]
- C. [10, 20, 30]
- D. [99, 20]

Bonne réponse : C. [10, 20, 30]

a et b désignent la même liste : l'ajout via b est visible dans a. c est une autre liste ; remplacer c[0] ne modifie pas a.

Python - Chapitre 2

Fonctions, collections et programmes fiables

Quiz de révision | 10 questions | Une seule bonne réponse par question

Question 5

Que montre `print(notes)` après le tri ?

```
notes = [12, 18, 15]
ordre = sorted(notes)
print(notes)
```

- A. [12, 15, 18]
- B. [12, 18, 15]
- C. None
- D. Une erreur

Bonne réponse : B. [12, 18, 15]

`sorted` crée une nouvelle liste, ici stockée dans `ordre`. `notes` garde son ordre initial. `notes.sort()` aurait modifié `notes`.

Question 6

Combien de lignes cette boucle affiche-t-elle ?

```
noms = ["Ada", "Linus", "Guido"]
notes = [18, 15]
for nom, note in zip(noms, notes):
    print(nom, note)
```

- A. 2
- B. 3
- C. 5
- D. Aucune : les longueurs différentes provoquent une erreur

Bonne réponse : A. 2

Sans l'option `strict=True`, `zip` s'arrête au plus court. La boucle affiche Ada 18 puis Linus 15 ; Guido n'a pas de note associée.

Python - Chapitre 2

Fonctions, collections et programmes fiables

Quiz de révision | 10 questions | Une seule bonne réponse par question

Question 7

Quelle valeur contient `comptes["chat"]` après la boucle ?

```
comptes = {}  
for mot in ["chat", "chien", "chat"]:  
    comptes[mot] = comptes.get(mot, 0) + 1  
print(comptes["chat"])
```

- A. 0
- B. 1
- C. 2
- D. 3

Bonne réponse : C. 2

`get(mot, 0)` fournit 0 quand la clé manque. Le premier chat fait passer son compteur à 1 ; le second le fait passer à 2.

Question 8

Quels identifiants restent dans le résultat ?

```
actifs = {"A", "B", "C"}  
contactes = {"B", "D"}  
print(sorted(actifs - contactes))
```

- A. ['B']
- B. ['A', 'C']
- C. ['A', 'B', 'C', 'D']
- D. ['D']

Bonne réponse : B. ['A', 'C']

La différence garde les éléments de actifs absents de contactes. Il reste A et C. `sorted` les présente dans une liste triée.

Python - Chapitre 2

Fonctions, collections et programmes fiables

Quiz de révision | 10 questions | Une seule bonne réponse par question

Question 9

Quelle liste obtient-on après ce nettoyage ?

```
noms = [" Ada ", "", "  "]
propres = [n.strip() for n in noms if n.strip() != ""]
print(propres)
```

- A. [' Ada ']
- B. ['Ada', '', '']
- C. []
- D. ['Ada']

Bonne réponse : D. ['Ada']

strip retire les espaces aux extrémités. Le filtre examine le texte nettoyé : la chaîne vide et celle qui ne contient que des espaces sont écartées.

Question 10

Quel message ce programme affiche-t-il ?

```
try:
    n = int("0")
    if n <= 0:
        raise ValueError("positif attendu")
    print("accepte")
except ValueError:
    print("refuse")
```

- A. refuse
- B. accepte
- C. 0
- D. Aucun : ValueError n'est pas traitée

Bonne réponse : A. refuse

La conversion de "0" réussit, mais 0 n'est pas strictement positif. raise interrompt le bloc try ; except traite l'erreur et affiche refuse.